Fraud Detection with PaySim: A Review of Learning Models

Introduction

This report explores the application of machine learning models to the **Synthetic Financial Datasets For Fraud Detection (PaySim)** dataset, aiming to identify effective strategies for detecting fraudulent transactions. We analyze existing approaches, highlighting impressive techniques and potential extensions for enhanced fraud detection.

Learning Tasks and Approaches in Fraud Detection

We reviewed several approaches to fraud detection using the PaySim dataset. Here are four notable examples:

Example 1: Jack T. Tai - XGBoost and SMOTE

- **Learning Task:** Binary classification of fraudulent transactions.
- **Approach:** Used XGBoost with SMOTE for class imbalance. Achieved **99.2% F1-score**.
- **Impressive Aspects and PaySim Application:**
  - **SMOTE:** Effectively addressed class imbalance, a key challenge in PaySim. Apply SMOTE or ADASYN before training models.

In [25]:
```
from imblearn.over_sampling import SMOTE, BorderlineSMOTE, ADASYN, SVMSMOTE
from imblearn.under_sampling import RandomUnderSampler
from imblearn.pipeline import Pipeline

from xgboost import XGBClassifier

from sklearn.model_selection import cross_val_score, GridSearchCV, RepeatedStratifiedKFold, cr
oss_validate
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix,plot_precis
ion_recall_curve
np.random.seed = 42
```

  - **Feature Engineering:** Correlation analysis, log transformation, one-hot encoding. Identify and handle skewed data and categorical features in PaySim similarly.

In [19]:
```
from sklearn.preprocessing import StandardScaler
scaler_df = comp_df.drop('Class',axis=1)
scaler_df = StandardScaler().fit_transform(scaler_df)
scaler_df = pd.DataFrame(scaler_df)
```

- **Link to Approach:** Jack T. Tai - XGBoost and SMOTE

Example 2: Rony Keller - Data Fraud Transactions Detector

- **Learning Task:** Binary classification of fraudulent transactions.
- **Approach:** Built a machine learning pipeline including data preprocessing, feature selection, and model training using Random Forest and Gradient Boosting.
- **Impressive Aspects and PaySim Application:**
  - **Comprehensive Data Preprocessing:** Addressed missing values, encoded categorical variables, and scaled numerical features, which is crucial for PaySim's raw data.
  - **Feature Selection:** Used feature selection techniques to reduce dimensionality and improve model efficiency.
  - **Model Evaluation:** Employed cross-validation and performance metrics like precision, recall, and F1-score.

8.1 splitting the Datas to train @ test : 80%-20%

```
# x_train, x_test, y_train, y_test = train_test_split(x_scaled, Y, train_size=0.80, random_state
=42)
randomState = 42
np.random.seed(3)
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size = 0.2, \
                                                    random_state = randomState)
```

```
weights = (Y == 0).sum() / (1.0 * (Y == 1).sum())
time.sleep(2)
clf = XGBClassifier(use_label_encoder=False, max_depth = 3, scale_pos_weight = weights, \
                n_jobs = 4, eval_metric='mlogloss')
probabilities = clf.fit(x_train, y_train).predict_proba(x_test)
print('Probability Score AUPRC = {}'.format(average_precision_score(y_test, \
                                            probabilities[:, 1])))
```

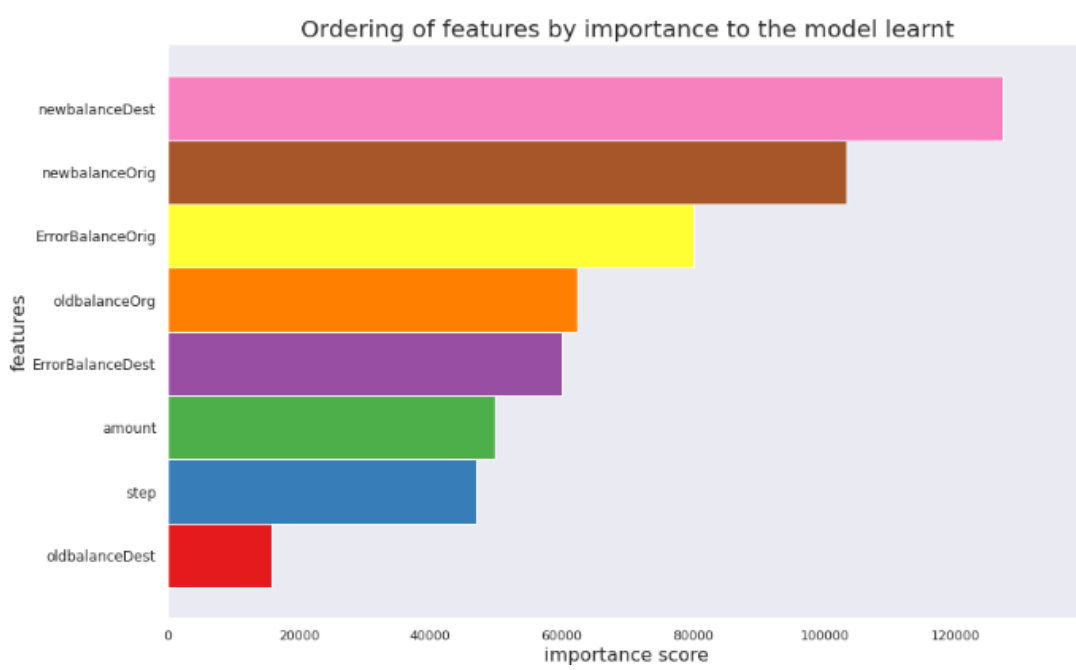
```
Probability Score AUPRC = 0.939592786221274
```

```
fig = plt.figure(figsize = (14, 9))
ax = fig.add_subplot(111)

colours = plt.cm.Set1(np.linspace(0, 1, 9))

ax = plot_importance(clf, height = 1, color = colours, grid = False, \
                     show_values = False, importance_type = 'cover', ax = ax);
for axis in ['top','bottom','left','right']:
            ax.spines[axis].set_linewidth(2)

ax.set_xlabel('importance score', size = 16);
ax.set_ylabel('features', size = 16);
ax.set_yticklabels(ax.get_yticklabels(), size = 12);
ax.set_title('Ordering of features by importance to the model learnt', size = 20);
```



- **Link to Approach:** [Rony Keller - Data Fraud Transactions Detector]

Example 3: Fraud Detection in Mobile Transactions

- **Learning Task:** Binary classification of fraudulent transactions.
- **Approach:** Implemented machine learning models including Logistic Regression and Random Forest.
- **Impressive Aspects and PaySim Application:**
  - **Feature Engineering:** Created new derived features based on transaction history to improve model accuracy.
  - **Anomaly Detection:** Integrated anomaly detection techniques to flag suspicious transactions more effectively.
  - **Scalability:** Considered how the model could be applied to large-scale real-time fraud detection systems.

```
model = {"Logistic Regression": LogisticRegression(),
         "KNN": KNeighborsClassifier(),
         "Random Forest": RandomForestClassifier(),
         "SGD": SGDClassifier()}

def fit_and_evaluate(models, X_train, X_test, y_train, y_test):
    """
    Fits and evaluates machine learning models.
    models: a dictionary with machine learning models to be used
    X_train: training data (no labels)
    X_test: testing data (no labels)
    y_train: training labels
    y_test: testing labels
    """
    # Create an empty dictionary for model scores
    model_scores = {}
    # Loop through models
    for name, model in models.items():
        model.fit(X_train, y_train)
        model_scores[name] = model.score(X_test, y_test)
    return model_scores
```

- **Link to Approach:** Fraud Detection in Mobile Transactions

Example 4: Smashell10 - LSTM-Based Anomaly Detection

- **Learning Task:** Anomaly detection in transaction sequences.
- **Approach:** Used LSTM neural networks to analyze sequential transaction data and detect anomalies.

- **Impressive Aspects and PaySim Application:**
  - **Deep Learning for Anomaly Detection:** Applied LSTM networks to model transaction sequences and identify unusual patterns.
  - **Sequence-Based Fraud Detection:** Leveraged the temporal nature of transactions to improve fraud detection accuracy.
  - **Potential for Real-Time Analysis:** This method could be extended for real-time fraud detection in financial systems.
-

```python
def build_autoencoder(input_dim):
    input_layer = layers.Input(shape=(input_dim,))
    encoded = layers.Dense(64, activation='relu')(input_layer)
    encoded = layers.Dense(32, activation='relu')(encoded)
    encoded = layers.Dense(16, activation='relu')(encoded)
    bottleneck = layers.Dense(8, activation='relu')(encoded)
    decoded = layers.Dense(16, activation='relu')(bottleneck)
    decoded = layers.Dense(32, activation='relu')(decoded)
    decoded = layers.Dense(64, activation='relu')(decoded)
    output_layer = layers.Dense(input_dim, activation='linear')(decoded)
    autoencoder = models.Model(inputs=input_layer, outputs=output_layer)
    return autoencoder

autoencoder = build_autoencoder(X_train.shape[1])
autoencoder.summary()

# Training the Autoencoder
autoencoder.compile(optimizer='adam', loss='mse')

early_stop = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

history = autoencoder.fit(
    X_train,
    X_train,
    epochs=10,
    batch_size=256,
    shuffle=True,
    validation_data=(X_val, X_val),
    callbacks=[early_stop],
    verbose=1
)

# Plotting training history
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss', color='blue')
plt.plot(history.history['val_loss'], label='Validation Loss', color='orange')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

```
# Computing reconstruction errors
reconstructions = autoencoder.predict(X_scaled)
mse = np.mean(np.power(X_scaled - reconstructions, 2), axis=1)
df_encoded['reconstruction_error'] = mse
df_encoded['isFraud'] = df['isFraud'].values

# Visualizing reconstruction errors
plt.figure(figsize=(10,6))
sns.histplot(df_encoded[df_encoded['isFraud'] == 0]['reconstruction_error'], bins=50, color='blue', label='Legitimate', kd
sns.histplot(df_encoded[df_encoded['isFraud'] == 1]['reconstruction_error'], bins=50, color='red', label='Fraudulent', kde=
plt.title('Reconstruction Error by Transaction Type')
plt.xlabel('Reconstruction Error')
plt.ylabel('Frequency')
plt.legend()
plt.show()

# Determining threshold
threshold = np.percentile(mse[df['isFraud'] == 0], 95)
print(f"Reconstruction error threshold: {threshold}")

# Classifying transactions
df_encoded['predicted_isFraud'] = df_encoded['reconstruction_error'] > threshold

# Evaluating the performance


y_true = df_encoded['isFraud']
y_pred = df_encoded['predicted_isFraud'].astype(int)

print(classification_report(y_true, y_pred))

cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Legitimate', 'Fraudulent'], yticklabels=['Legitimate', 'F
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix')
plt.show()
```

Model: "functional"

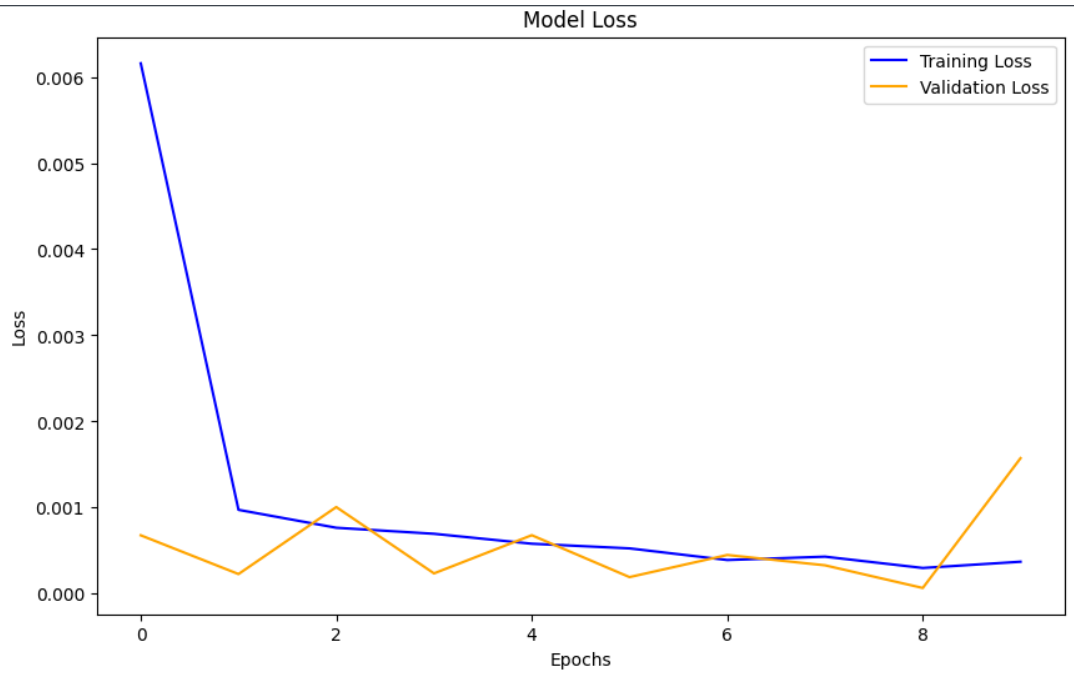
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 11)	0
dense (Dense)	(None, 64)	768
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 8)	136
dense_4 (Dense)	(None, 16)	144
dense_5 (Dense)	(None, 32)	544
dense_6 (Dense)	(None, 64)	2,112
dense_7 (Dense)	(None, 11)	715

Total params: 7,027 (27.45 KB)

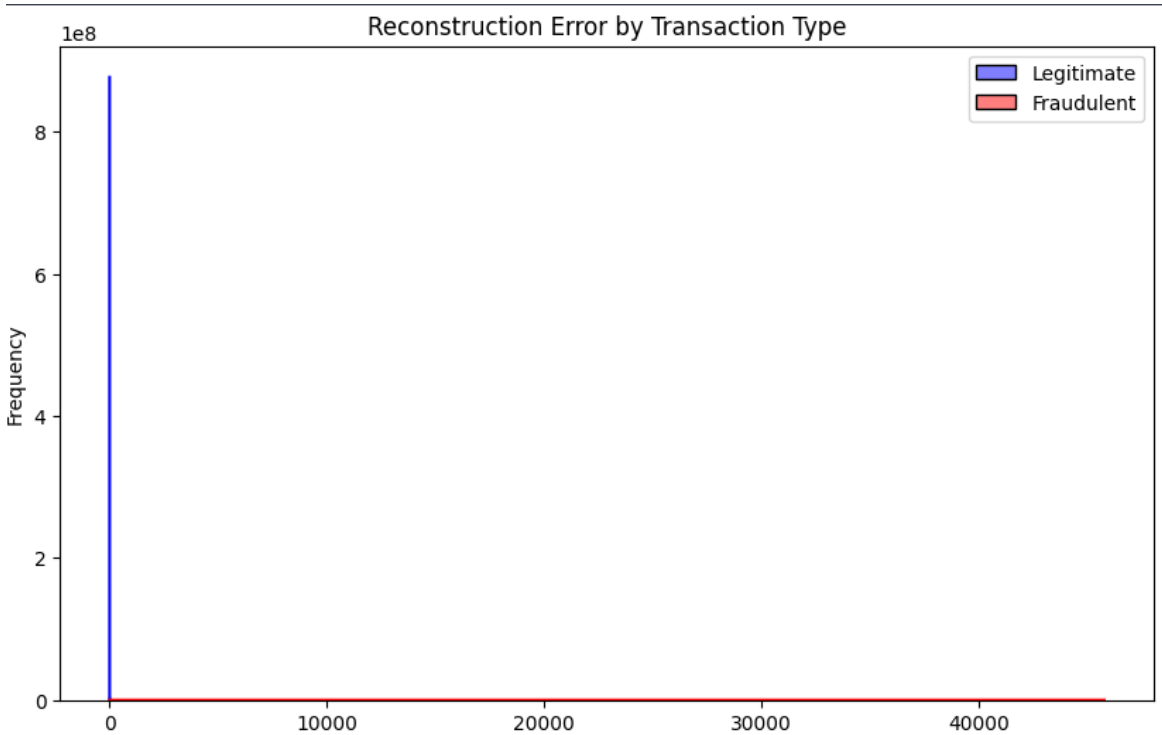
Trainable params: 7,027 (27.45 KB)

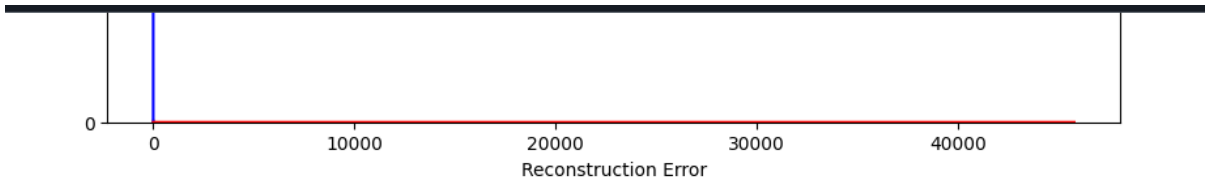
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 69s 3ms/step - loss: 0.0250 - val_loss: 6.7837e-04
Epoch 2/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 65s 3ms/step - loss: 0.0012 - val_loss: 2.2852e-04
Epoch 3/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 76s 3ms/step - loss: 8.1900e-04 - val_loss: 0.0010
Epoch 4/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 86s 3ms/step - loss: 6.8270e-04 - val_loss: 2.3620e-04
Epoch 5/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 81s 3ms/step - loss: 6.7281e-04 - val_loss: 6.7959e-04
Epoch 6/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 87s 3ms/step - loss: 5.3946e-04 - val_loss: 1.9300e-04
Epoch 7/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 81s 3ms/step - loss: 4.4604e-04 - val_loss: 4.5004e-04
Epoch 8/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 66s 3ms/step - loss: 4.5347e-04 - val_loss: 3.3005e-04
Epoch 9/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 87s 4ms/step - loss: 2.9637e-04 - val_loss: 6.6808e-05
Epoch 10/10
19858/19858 ━━━━━━━━━━━━━━━━━━━━ 77s 3ms/step - loss: 4.8199e-04 - val_loss: 0.0016
```



- 198832/198832 ━━━━━━━━━━━━━━━━━━━━ 317s 2ms/step

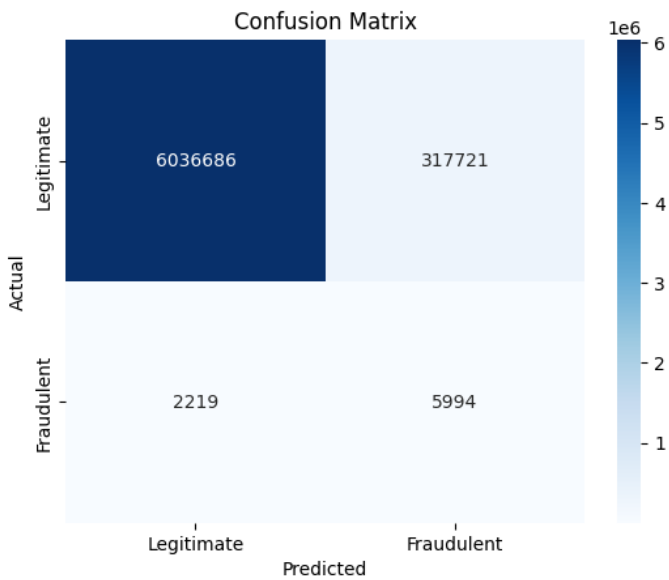





```
Reconstruction error threshold: 0.00014429343553252195
              precision    recall  f1-score   support

           0       1.00      0.95      0.97   6354407
           1       0.02      0.73      0.04      8213

    accuracy                           0.95   6362620
   macro avg       0.51      0.84      0.51   6362620
weighted avg       1.00      0.95      0.97   6362620
```



-
- **Link to Approach:** Smashell10 - Anomaly Detection in Financial Transactions

Conclusion

This review highlights effective approaches for fraud detection using PaySim. Techniques like **SMOTE, feature engineering, and anomaly detection** can build accurate models. Future work can explore **real-time fraud detection and advanced anomaly detection methods** to improve results.

References

1. **Jack T. Tai:** XGBoost and SMOTE
2. **Rony Keller:** Data Fraud Transactions Detector
3. **Fraud Detection in Mobile Transactions:** Mobile Fraud Detection

4. **Smashell10 - LSTM-Based Anomaly Detection** Anomaly-Detection-in-Financial-Transactions